Hybrid News Ingestion & Trigger Architecture

Cost-Efficient, Scalable and Real-Time Enough

The biggest challenge in a personalized news platform is balancing **freshness**, **personalization**, and **cost**.

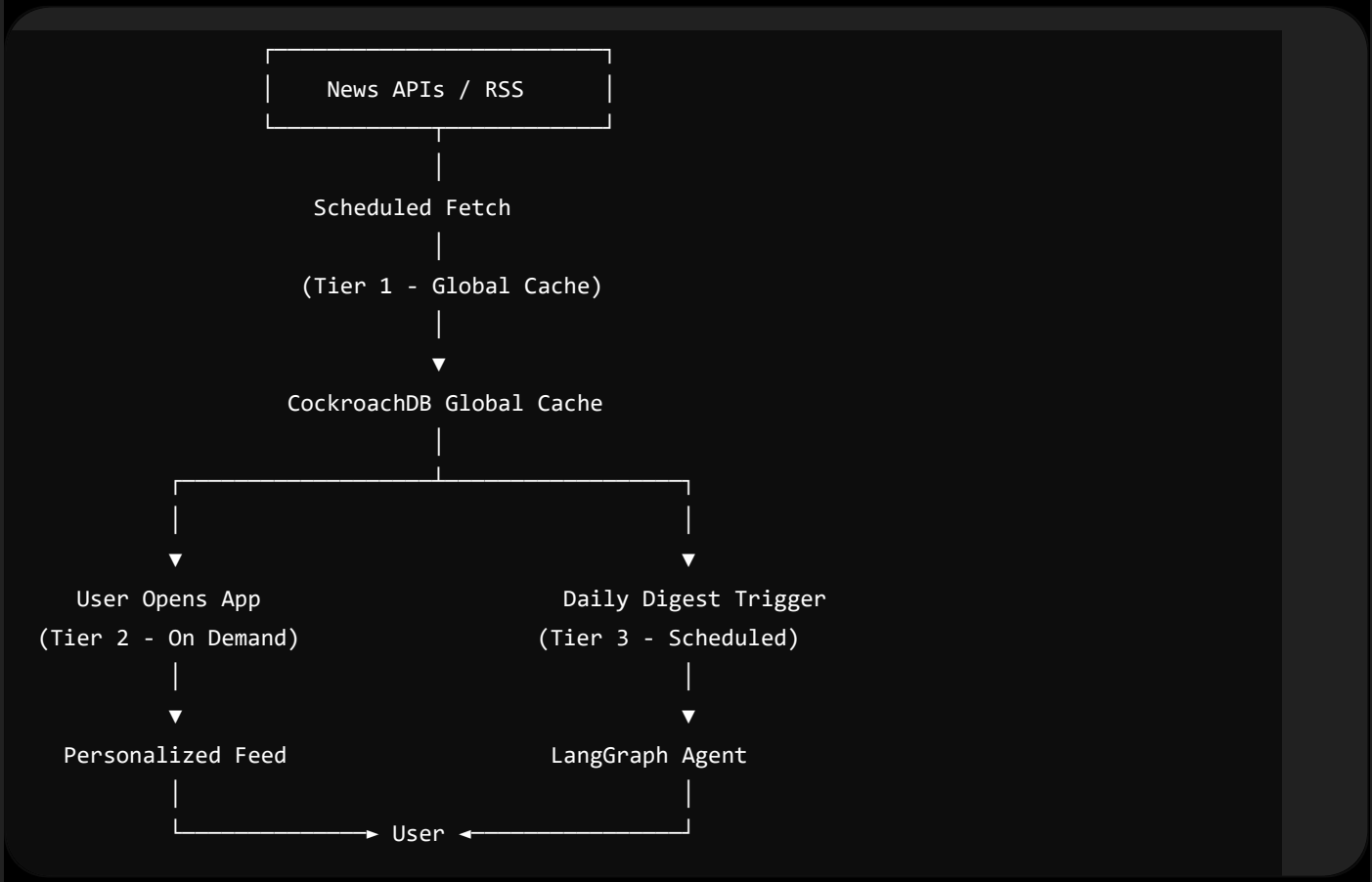
A naïve solution would continuously scrape all news sources every few minutes. While this keeps the database up to date, it results in:

- High API usage
- Frequent AWS Lambda executions
- Increased embedding generation costs
- Storing many unused articles
- Higher CockroachDB storage and compute costs

Instead, AmpliNews adopts a **Hybrid Pull Architecture** that combines scheduled ingestion, intelligent caching, and on-demand refreshing.

Overall Idea

The architecture divides news processing into three independent tiers.



The key idea is that **not every operation needs to happen continuously**.

Tier 1 — Global News Cache

Purpose

Maintain a lightweight cache of the latest important news from around the world.

Instead of downloading thousands of articles every hour, the system periodically fetches only the most relevant headlines from each category.

Example categories:

- Politics
- Technology
- Business
- Sports
- Health
- Science
- Entertainment

For each category only the **Top 10–20 latest articles** are fetched.

Example

```
Technology     → 20 articles
Politics       → 20 articles
Business       → 20 articles
Sports         → 20 articles
Health         → 20 articles
```

Approximately

```
100–150 articles
```

per ingestion cycle.

Trigger

AWS EventBridge

```
Every 30–60 minutes
```

↓

AWS Lambda

↓

Fetch latest articles

↓

Store in CockroachDB

Stored Information

Each article contains

- Title
- Content
- Source
- Publication time
- Bias score
- Credibility score
- Topic
- Embedding (if generated)

This collection forms the **Global News Cache**, which is shared among all users.

Why This Works

Instead of every user individually querying external news APIs, all users reuse the same cached articles.

Benefits:

- Lower API usage
- Lower AWS execution time
- Faster responses
- Shared vector database

Tier 2 — Cache Refresh on Demand

Purpose

Keep news fresh without constantly polling external APIs.

Instead of refreshing continuously, the backend checks whether cached data is still sufficiently recent whenever a user requests news.

Workflow

User opens application.

↓

FastAPI receives request.

↓

Checks newest cached article timestamp.

```
Latest Article

↓

Is cache fresh?
```

Scenario 1

Cache age

```
15 minutes
```

Result

```
Use existing cache.
```

No external API call.

Scenario 2

Cache age

```
2 hours
```

Result

```
Fetch latest news
ONLY for required topics.
```

Example

User interests

```
Artificial Intelligence

Cloud Computing

Politics
```

Backend fetches only

```
Technology

Politics
```

instead of every news category.

Why It Saves Resources

Instead of refreshing all categories for every user,

the backend refreshes only

- requested categories
- stale data

This approach significantly reduces

- News API calls
- Lambda executions
- Embedding generation

while keeping frequently viewed topics current.

Tier 3 — Scheduled Daily Digest

The personalized digest does not require real-time execution.

It runs once every day.

Trigger

AWS EventBridge

```
8:00 AM
```

↓

AWS Lambda

↓

LangGraph Agent

↓

Generate Digest

↓

Email via AWS SES

The agent retrieves

- User profile
- Reading history
- User interests
- Bias trends
- Latest cached articles

and generates

- Personalized recommendations
- Opposing viewpoints
- Reading insights
- Diversity score

Since this workflow executes only once daily per user, infrastructure costs remain predictable.

Fetching Strategy

Instead of requesting

```
Everything
```

the system performs **targeted retrieval**.

Global Fetch

The scheduler periodically retrieves

```
Top headlines

Latest trending stories

Breaking news

Major world events
```

This creates a common dataset used by everyone.

Personalized Fetch

When needed,

the agent derives search queries directly from user interests.

Example

User Profile

```
AI

Machine Learning

AWS

Cloud Computing
```

Generated API queries

```
Artificial Intelligence

LLMs

AWS

Cloud

Generative AI
```

Only these articles are fetched and indexed.

This keeps API usage proportional to actual user interests.

Resource Optimization

The architecture minimizes compute costs through several optimizations.

1. Shared Global Cache

One cached article can serve hundreds of users.

No duplicate downloads.

2. On-Demand Refresh

News is refreshed only when

- cache becomes stale
- user requests it

instead of continuous polling.

3. Category-Based Fetching

Only categories that are actually required are downloaded.

Example

```
Politics

Technology
```

instead of

```
Politics

Sports

Entertainment

Finance

Health

Science
```

4. Lazy Embedding Generation

Generating embeddings is one of the most expensive operations.

Instead of embedding every fetched article immediately,

the system delays embedding until

- the article becomes trending,
- it is selected for recommendations,
- or a user requests content from that topic.

Unused articles never consume embedding resources.

5. Deduplication

Before storing,

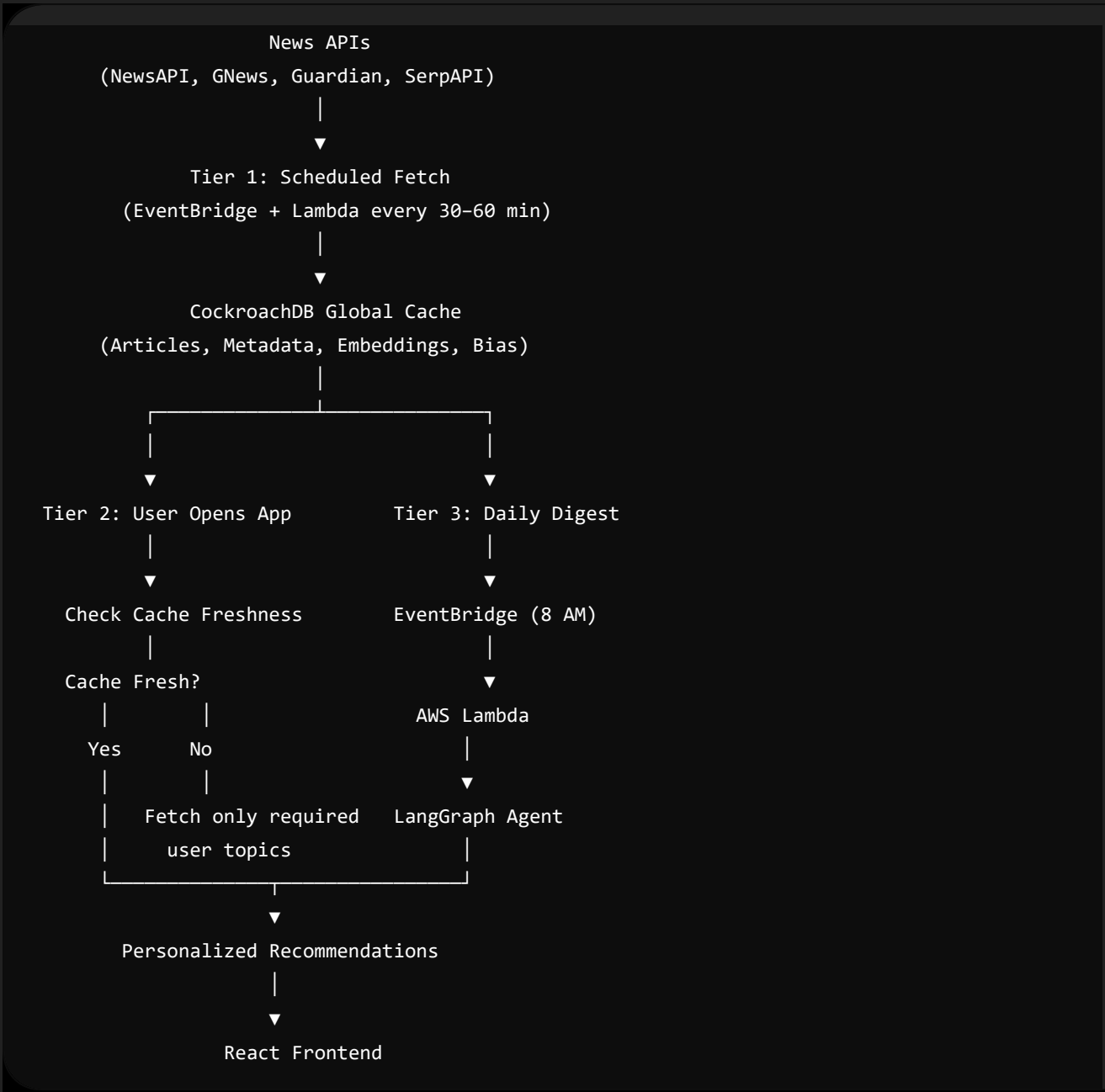
the backend checks

- article URL
- title similarity
- publication timestamp

to avoid processing duplicate news from multiple providers.

Hybrid Pull Architecture

The architecture combines **scheduled ingestion** with **on-demand retrieval**, achieving a balance between freshness and cost.



Advantages of the Hybrid Architecture

- **Cost-efficient:** Scheduled ingestion is lightweight, and expensive operations run only when necessary.
- **Scalable:** A single global cache serves all users while personalization happens dynamically.
- **Fresh:** Frequently accessed topics are refreshed on demand rather than waiting for the next scheduled job.
- **Responsive:** Most user requests are answered directly from CockroachDB, reducing latency.
- **Hackathon-aligned:** Demonstrates CockroachDB as the persistent memory layer while showcasing efficient AWS usage through EventBridge, Lambda, and intelligent trigger design.